accenture

Cloud Platform Deployment

AWS — EC2, S3, Lambda, and CloudFront

Chapter Overview

01 AWS Fundamentals and Global Infrastructure

02 IAM and Security

03 EC2 — Virtual Servers

04 S3 and CloudFront — Static Hosting

05 Lambda and Serverless

06 Architecture Decision Guide

Copyright © 2026 Accenture. All rights reserved.

AWS Fundamentals

Section 1



What is AWS?

Cloud Computing Model

- On-demand computing resources — pay only for what you use
- No upfront hardware — provision servers in minutes, not weeks
- 200+ managed services spanning compute, storage, networking, and databases

Service Models

- IaaS (e.g., EC2) — you manage the OS, runtime, and app
- PaaS (e.g., Elastic Beanstalk) — platform manages infrastructure
- FaaS (e.g., Lambda) — run code without managing any servers
- SaaS (e.g., Salesforce, Microsoft 365) — fully managed application; you only configure and use the software

Copyright © 2026 Accenture. All rights reserved.

AWS Global Infrastructure



39 Regions • 123 Availability Zones • 750+ CloudFront Points of Presence + 15 Regional Edge Caches

Copyright © 2026 Accenture. All rights reserved.

Regions, AZs, and Edge Locations

Regions

- Geographic locations (us-east-1, eu-west-1, etc.) — choose based on user proximity and compliance
- Not all services are available in all regions — check before deploying

Availability Zones (AZs)

- Each AZ is one or more discrete data centers with redundant power and networking
- Deploy across multiple AZs for high availability — if one AZ fails, others continue

Edge Locations

- 750+ edge locations globally + regional edge caches — serve content close to end users



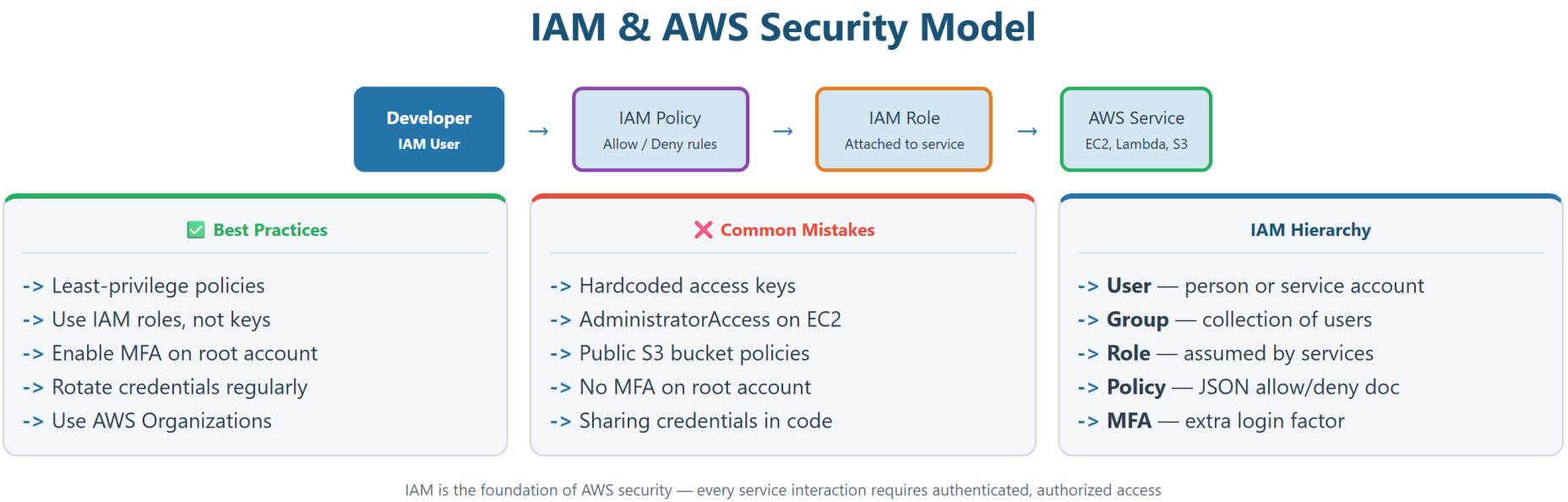
Copyright © 2026 Accenture. All rights reserved.

IAM and Security

Section 2



IAM & AWS Security Model



Copyright © 2026 Accenture. All rights reserved.

IAM — Identity and Access Management

Core Components

- Users — individual people or service accounts with long-term credentials
- Groups — collections of users sharing the same permissions
- Roles — temporary credentials assumed by services (EC2, Lambda, etc.)
- Policies — JSON documents defining allow/deny rules for actions and resources

Least Privilege Principle

- Grant only the minimum permissions required — start with deny-all and add as needed
- Use IAM roles for services instead of embedding access keys in code

Copyright © 2026 Accenture. All rights reserved.

IAM Policy Example

```
// IAM policy — least-privilege S3 access
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:PutObject"
    ],
    "Resource": "arn:aws:s3:::my-app-bucket/*"
  }]
}

// Attach to an IAM role, NOT a user
// EC2 instances and Lambda functions assume the role
```



Copyright © 2026 Accenture. All rights reserved.

VPC Networking Essentials

Virtual Private Cloud (VPC)

- Your own isolated network within AWS — you control IP ranges, subnets, and routing
- Every EC2 instance, RDS database, and Lambda (with VPC config) runs inside a VPC

Subnet Types

- Public subnet — has route to Internet Gateway, for web servers and load balancers
- Private subnet — no direct internet access, for databases and internal services

Security Layers

- Security Groups — stateful instance-level firewall (allow rules only)
- NACLs — stateless subnet-level firewall (allow and deny rules)

Copyright © 2026 Accenture. All rights reserved.

EC2 — Virtual Servers

Section 3



EC2 — Elastic Compute Cloud

What EC2 Provides

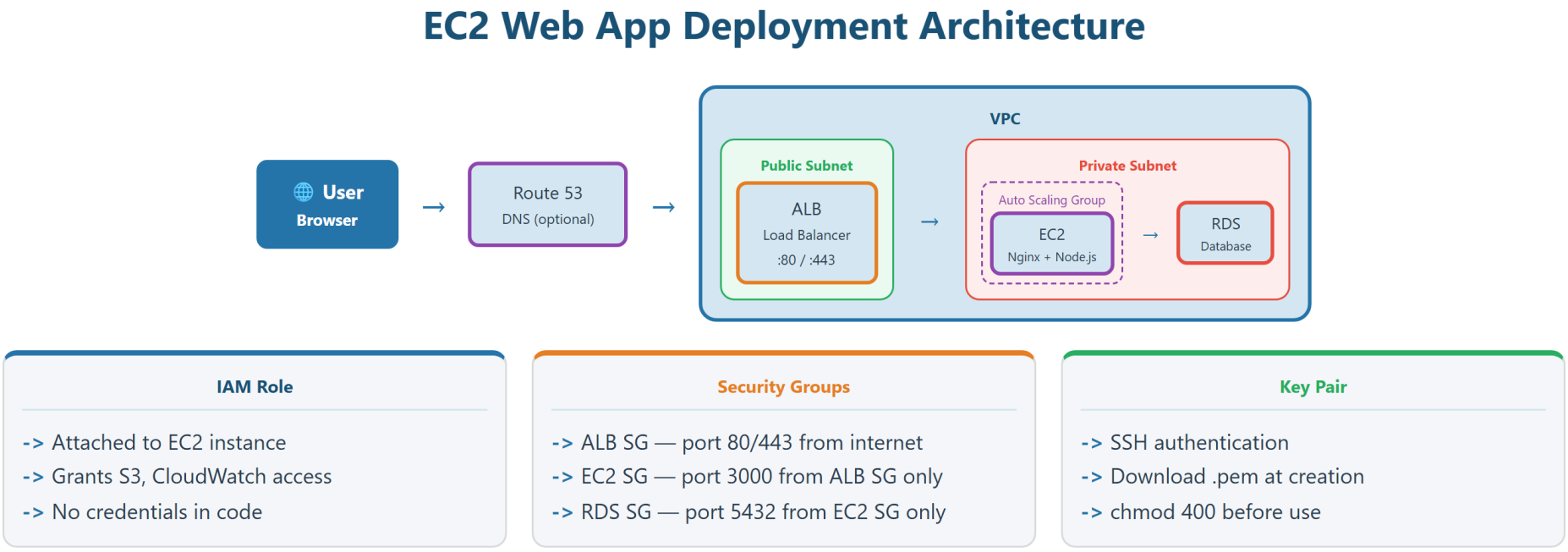
- Virtual machines that run your applications — choose CPU/GPU, memory, storage, and OS
- Full control: install any software, configure networking, attach storage volumes

Key Components

- AMI — Amazon Machine Image (OS template: Amazon Linux 2023, Ubuntu 22.04)
- Instance type — t3.micro (free tier), t3.small, m6i.large based on workload
- Security Group — firewall controlling inbound (SSH port 22, HTTP port 80) and outbound
- Key Pair — SSH key for secure instance access (download .pem at creation)

Copyright © 2026 Accenture. All rights reserved.

EC2 Web App Deployment Architecture



Copyright © 2026 Accenture. All rights reserved.

EC2 Production Deployment

```
# Connect to EC2 instance (Amazon Linux 2023)
ssh -i my-key.pem ec2-user@54.123.45.67

# Create a non-root app user
sudo useradd -m -s /bin/bash appuser

# Install Node.js 22 LTS (Amazon Linux)
sudo dnf install -y nodejs22

# Install PM2 globally for appuser
sudo -u appuser npm install -g pm2

# Deploy the application
sudo -u appuser bash -c '
  cd /home/appuser
  git clone https://github.com/org/my-api.git
  cd my-api && npm ci --omit=dev
'

# Start with PM2 and configure auto-restart
sudo -u appuser pm2 start server.js --name my-api
sudo -u appuser pm2 save
sudo env PATH=$PATH pm2 startup systemd -u appuser –hp /home/appuser
```



Copyright © 2026 Accenture. All rights reserved.

Nginx Reverse Proxy for Node.js

```
# Install and configure Nginx as reverse proxy
sudo dnf install -y nginx

# /etc/nginx/conf.d/my-api.conf
server {
    listen 80;
    server_name my-api.example.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# Test config and start
sudo nginx -t && sudo systemctl enable --now nginx
```



Copyright © 2026 Accenture. All rights reserved.

EC2 Operational Concerns

Process Management

- Use PM2 or systemd to keep your Node.js app running after crashes and reboots
- Put Nginx in front as a reverse proxy — handles SSL termination, static files, load balancing

Monitoring and Maintenance

- CloudWatch agent sends CPU, memory, and disk metrics — set alarms for high utilization
- Enable CloudWatch Logs agent to stream application logs from the instance
- You are responsible for OS security patches — enable automatic updates or schedule them



Copyright © 2026 Accenture. All rights reserved.

S3 and CloudFront

Section 4



S3 — Simple Storage Service

Object Storage

- Store files (objects) in buckets — virtually unlimited storage capacity
- 11 nines of durability (99.999999999%) — your data is replicated across multiple AZs

Static Website Hosting

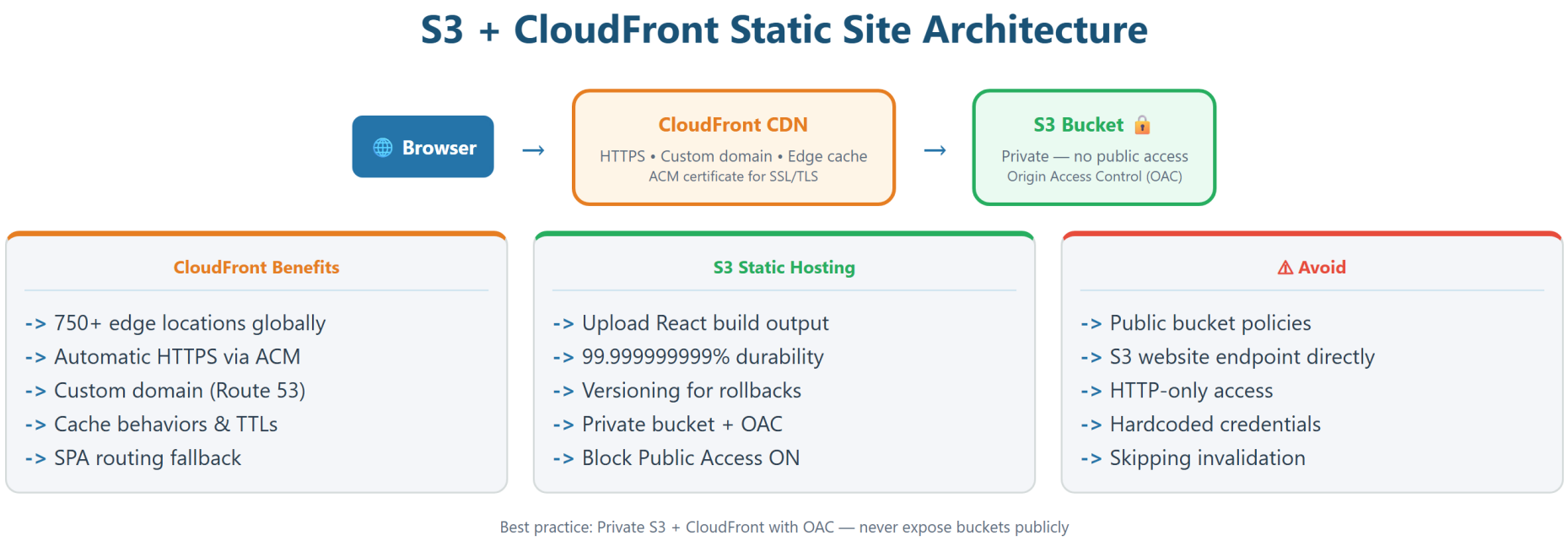
- Upload HTML, CSS, JS, and images — S3 serves them like a web server
- Perfect for React builds: npm run build produces static files ready for S3

Security Best Practice

- Keep the bucket private — use CloudFront with Origin Access Control (OAC) instead
- Enable Block Public Access on all buckets by default

Copyright © 2026 Accenture. All rights reserved.

S3 + CloudFront Static Site Architecture



Copyright © 2026 Accenture. All rights reserved.

CloudFront CDN

How It Works

- CloudFront caches your content at 750+ edge locations worldwide + regional edge caches
- Users get content from the nearest edge — reduces latency from seconds to milliseconds

Origin Access Control

- OAC lets CloudFront access a private S3 bucket — no public bucket policy needed
- S3 bucket policy allows only the CloudFront distribution's service principal

HTTPS and Custom Domains

- AWS Certificate Manager (ACM) provides free public SSL/TLS certificates
- Attach a custom domain via Route 53 — your-app.company.com

Copyright © 2026 Accenture. All rights reserved.

Deploy React to S3 + CloudFront

```
# Build the React app
npm run build

# Sync to PRIVATE S3 bucket (no public access)
aws s3 sync build/ s3://my-app-bucket \
  --delete --cache-control max-age=31536000

# Upload index.html with no-cache (always fresh)
aws s3 cp build/index.html s3://my-app-bucket/ \
  --cache-control no-cache

# Invalidate CloudFront cache for immediate update
aws cloudfront create-invalidation \
  --distribution-id E1ABC2DEF3GH \
  --paths '/index.html'

# Site is served via CloudFront HTTPS URL:
# https://d123abc.cloudfront.net
```



Copyright © 2026 Accenture. All rights reserved.

CloudFront Cache Management

Cache Behaviors

- Define TTLs per path pattern — long cache for /static/*, no-cache for index.html
- Versioned filenames (main.a1b2c3.js) allow aggressive caching with instant updates

Invalidation

- Force edge caches to re-fetch from S3 — first 1,000 invalidation paths/month are free
- Prefer versioned filenames over frequent invalidations for cost efficiency

SPA Routing

- Configure a custom error response: 404 → /index.html with 200 status
- React Router handles client-side routing — CloudFront just serves the shell

Copyright © 2026 Accenture. All rights reserved.

Lambda and Serverless

Section 5



Lambda Concepts

Functions as a Service

- Write a function, deploy it — AWS runs it only when triggered
- No servers to manage — AWS handles provisioning, scaling, and patching

Execution Model

- Each invocation runs in an isolated environment with configurable memory (128 MB–10 GB)
- Maximum execution time: 15 minutes per invocation
- Stateless — use DynamoDB, S3, or ElastiCache for persistent data

Pricing

- Pay per request ($0.20/1M) + compute time — free tier includes 1M requests/month

Copyright © 2026 Accenture. All rights reserved.

Lambda Triggers and Integrations

Synchronous Triggers

- API Gateway — HTTP requests invoke the function and return the response to the client
- Application Load Balancer — route traffic to Lambda like a regular target

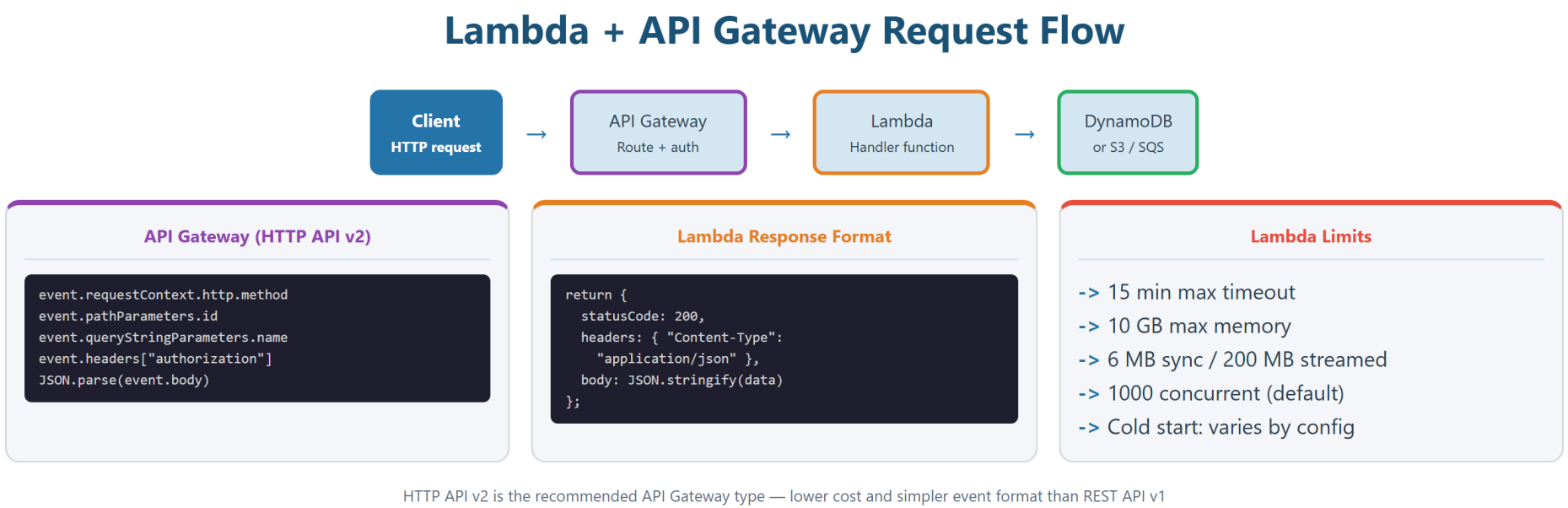
Asynchronous Triggers

- S3 events — file uploads trigger image processing, virus scanning, or ETL
- DynamoDB Streams — database changes trigger downstream updates
- EventBridge — schedule functions with cron expressions (e.g., daily reports)
- SQS — process queued messages with built-in retry and dead-letter queues



Copyright © 2026 Accenture. All rights reserved.

Lambda + API Gateway Request Flow



HTTP API v2 is the recommended API Gateway type — lower cost and simpler event format than REST API v1

Copyright © 2026 Accenture. All rights reserved.

Lambda Handler — API Gateway

```
// Lambda handler for HTTP API (API Gateway v2 format)
import { DynamoDBClient, GetItemCommand } from '@aws-sdk/client-dynamodb';

const db = new DynamoDBClient({});

export const handler = async (event) => {
  try {
    const userId = event.pathParameters?.id;
    if (!userId) {
      return { statusCode: 400, body: JSON.stringify({ error: 'Missing user ID' }) };
    }

    const { Item } = await db.send(new GetItemCommand({
      TableName: process.env.USERS_TABLE,
      Key: { userId: { S: userId } },
    }));

    if (!Item) return { statusCode: 404, body: JSON.stringify({ error: 'Not found' }) };
    return { statusCode: 200, body: JSON.stringify(Item) };
  } catch (err) {
    console.error('Handler error:', err);
    return { statusCode: 500, body: JSON.stringify({ error: 'Internal server error' }) };
  }
};
```



Copyright © 2026 Accenture. All rights reserved.

Lambda Limitations and Cold Starts

Key Constraints

- 15-minute maximum execution time — not suitable for long-running processes
- 10 GB max memory / 512 MB–10 GB ephemeral storage
- Payload: 6 MB sync request/response, 200 MB streamed sync, 1 MB async
- 1,000 concurrent executions per region (default, can request increase)

Cold Starts

- Cold starts vary widely (sub-100ms to >1s depending on runtime, package size, and VPC usage)
- Subsequent invocations reuse the warm container — near-instant startup
- Provisioned Concurrency keeps functions warm — eliminates cold starts at extra cost

Copyright © 2026 Accenture. All rights reserved.

Monitoring with CloudWatch

CloudWatch Logs

- Lambda automatically streams console.log output to CloudWatch Log Groups
- EC2 requires installing the CloudWatch agent to ship application logs

CloudWatch Metrics and Alarms

- Built-in metrics: invocation count, error rate, duration, throttles (Lambda)
- EC2 metrics: CPU, network, disk — custom metrics via agent for memory
- Set alarms to notify via SNS (email/Slack) when error rate or latency spikes

Cost Monitoring

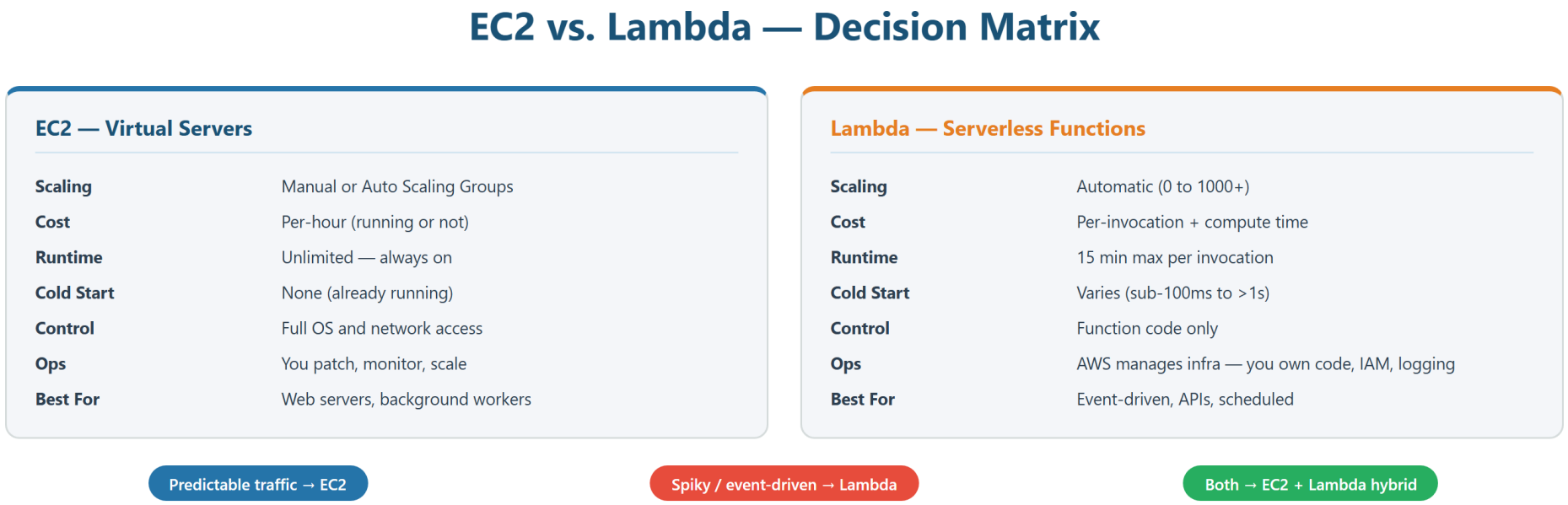
- AWS Budgets for thresholds and alerts
- Cost Anomaly Detection for unexpected spikes
- Cost Explorer for spend analysis by service, account, and tag

Copyright © 2026 Accenture. All rights reserved.

Architecture Decision Guide

Section 6

EC2 vs. Lambda — Decision Matrix



Copyright © 2026 Accenture. All rights reserved.

When to Use EC2

Long-Running Processes

- Web servers, WebSocket connections, and background workers that run continuously
- Applications exceeding Lambda's 15-minute execution limit

Full Control Needed

- Custom OS configuration, specialized dependencies, or GPU workloads
- Applications requiring persistent local disk or in-memory state

Cost Predictability

- Sustained high-traffic workloads — Reserved Instances or Savings Plans reduce cost 40-70%
- Lambda can be more expensive than EC2 at sustained high request volumes



Copyright © 2026 Accenture. All rights reserved.

When to Use Lambda

Event-Driven Processing

- API endpoints, file processing, scheduled tasks, and webhook handlers
- Functions that complete in seconds to minutes, not hours

Variable or Spiky Traffic

- Scales from zero to thousands of concurrent invocations automatically
- Pay nothing when there is no traffic — ideal for dev/staging environments

Zero Ops Desired

- No OS patching, no capacity planning, no server monitoring
- Caveat: you still own function code quality, logging, and IAM configuration

Copyright © 2026 Accenture. All rights reserved.

Deployment Automation with CI/CD

Why Automate?

– Manual SSH deploys are error-prone — CI/CD pipelines ensure consistent, repeatable releases

CodePipeline / GitHub Actions

– AWS CodePipeline — native AWS CI/CD: Source → Build → Deploy stages

– GitHub Actions — use aws-actions/configure-aws-credentials with OIDC (no stored keys)

Infrastructure as Code

– AWS CloudFormation or CDK define infrastructure in code — version and review changes

– SAM (Serverless Application Model) simplifies Lambda + API Gateway deployments

Copyright © 2026 Accenture. All rights reserved.

GitHub Actions — Deploy to S3 + CloudFront

```yaml
# .github/workflows/deploy.yml
name: Deploy to AWS
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write    # OIDC token for AWS auth
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/deploy
          aws-region: us-east-1
      - run: npm ci && npm run build
      - run: aws s3 sync build/ s3://my-app-bucket --delete
      - run: |
          aws cloudfront create-invalidation \
            --distribution-id ${{ secrets.CF_DIST_ID }} \
            --paths '/index.html'
```



Copyright © 2026 Accenture. All rights reserved.

Real-World Architecture Patterns

Static Frontend + Serverless API

- React on S3/CloudFront → API Gateway → Lambda → DynamoDB
- Fully serverless, scales to zero, minimal operational overhead

Full-Stack on EC2

- Nginx → Node.js/Express → RDS (PostgreSQL) on a single or multiple instances
- Use Auto Scaling Groups and Application Load Balancer for production traffic

Hybrid Architecture

- EC2 for the main API server + Lambda for event-driven tasks (image processing, emails)
- Most production AWS deployments use a mix of both compute models

Copyright © 2026 Accenture. All rights reserved.

Key Takeaways

Key Takeaways

1. AWS Regions contain multiple AZs — deploy across AZs for high availability
2. IAM roles with least-privilege policies — never embed access keys in code
3. EC2 for long-running servers: use Nginx, PM2 startup, and CloudWatch monitoring
4. S3 + CloudFront with OAC — keep buckets private, cache at the edge, HTTPS via ACM
5. Lambda for event-driven functions: scales to zero, 15-min limit, mind cold starts
6. Automate deployments with CI/CD — GitHub Actions + OIDC for keyless AWS access

Copyright © 2026 Accenture. All rights reserved.